

Vigil — Software Design Document

How twenty-five user stories and six wireframes became a modular, maintainable system

Name	Vatsal Agrawal	Registration	24BAI1355
Programme	B.Tech CSE (AI & ML), VIT Chennai	Date	24 September 2026
Repository	github.com/vatsal-agra/vigil	Design folder	/docs/design

Vigil is a self-hosted uptime and API monitor. A background worker probes each URL on a schedule, records every response, opens an incident after repeated failures, closes it on recovery with a one-sentence explanation, and publishes a public status page. Review 1 defined the product: vision, 25 user stories, MoSCoW priorities, six wireframes and a system diagram. This document explains how that was turned into structure. It covers which principles shaped the code, which architectural style was chosen and why, how the UI was made easy to use, and how the design absorbs future change. Every snippet is taken from the repository, with its file and line range below it.

1 Design Principles Applied

1.1 Abstraction — callers state *what*, never *how*

Each boundary hides its mechanism behind a small interface. On the frontend, no page ever calls `fetch`, reads the token, sets headers or handles an expired session. All four pages call one generic function:

```
export async function api<T>(path: string, init: RequestInit = {}): Promise<T> {
  const headers: Record<string, string> = { "Content-Type": "application/json", ...(init.headers ?? {}) };
  const t = token();
  if (t) headers.Authorization = `Bearer ${t}`;

  const res = await fetch(`${API}${path}`, { ...init, headers, cache: "no-store" });
  if (res.status === 401) { // one place decides what "signed out" means
    clearSession();
    window.location.href = "/login";
    throw new Error("Session expired.");
  }
  if (!res.ok) throw new Error((await res.json().catch(() => ({}))).detail ?? `Request failed (${res.status}).`);
  return res.status === 204 ? (undefined as T) : ((await res.json()) as T);
}
```

frontend/lib/api.ts, lines 53–72 (condensed)

The backend treats Redis the same way. `cache.py` exposes three verbs and absorbs every Redis failure, so to its callers the cache is *optional*: if Redis is down, reads fall through to PostgreSQL rather than erroring.

```
def get_json(key: str) -> Any | None:
    try:
        raw = client().get(key)
    except redis.RedisError:
        return None # a cache outage looks exactly like a cache miss
    return json.loads(raw) if raw else None

def invalidate(pattern: str) -> None:
    try:
        conn = client()
        for key in conn.scan_iter(match=pattern, count=200):
            conn.delete(key)
    except redis.RedisError:
        pass
```

Why it matters: moving the session token from `localStorage` to an `httpOnly` cookie changes one file, `api.ts`. Swapping Redis for another store changes one file, `cache.py`. No page and no router has to be touched.

1.2 Modularity — one file, one job

The code is split into four layers, and every module sits in exactly one of them (Figure 2). Dependencies only point downward: routers use services, services use models, and nothing imports upward. The probe worker is a second *process*, not a second codebase. `worker.py` is a 40-line loop over the same `probe.py`, `models.py` and `config.py` that the API uses:

```
def main() -> None:
    wait_for_db()
    log.info("worker online, cycle every %ss", settings.probe_interval_seconds)
    while True:
        db = SessionLocal()
        try:
            checked = run_cycle(db)          # all logic lives in app/probe.py
            if checked:
                log.info("cycle complete, %s monitor(s) checked", checked)
        except Exception:
            log.exception("cycle failed, continuing")    # one bad cycle never kills the worker
        finally:
            db.close()
        time.sleep(settings.probe_interval_seconds)
```

backend/worker.py, lines 27–40

Module	Single responsibility	User stories
<code>routers/auth.py</code>	Register and sign in; issue a JWT	US-01, US-02
<code>routers/monitors.py</code>	Monitor CRUD, response-time series, ownership check	US-05 – US-12
<code>routers/incidents.py</code>	Incident feed, open-only filter, acknowledge	US-15, US-18
<code>routers/status.py</code>	Unauthenticated, cached public status feed	US-16, US-17
<code>probe.py</code>	Checking engine and incident state machine	US-13, US-14, US-21
<code>services.py</code>	Shared read queries: 24-hour rollup, open incident, unique slug	US-08, US-14
<code>security.py</code>	Password hashing, JWT issue/verify, <code>current_user</code> dependency	US-01 – US-04
<code>cache.py</code> / <code>db.py</code> / <code>config.py</code>	Redis access / sessions / every tunable from environment variables	US-23, US-24

1.3 High cohesion — each function does one step of one job

`probe.py` holds only the checking engine, split into three functions that each do one step: `due_monitors()` decides *which* monitors to check, `probe_once()` *measures* one of them and returns a `Check` without touching the database, and `apply_result()` performs the *state transition*. Because measuring never writes, `probe_once` can be unit-tested with a fake HTTP client. All of the transition logic is in one place:

```
def apply_result(db: Session, monitor: Monitor, check: Check) -> None:
    monitor.last_checked_at = datetime.now(timezone.utc)
    if check.ok:
        monitor.consecutive_failures = 0
        monitor.current_status = "degraded" if check.latency_ms > 2000 else "up"
        incident = open_incident(db, monitor.id)
        if incident is not None:
            # recovery closes the incident
            incident.resolved_at = datetime.now(timezone.utc)
            incident.summary = f"{monitor.name} recovered after {minutes:.0f} min. Trigger: {incident.cause}."
    else:
        monitor.consecutive_failures += 1
        if monitor.consecutive_failures >= settings.failure_threshold: # flap guard: 2, not 1
            monitor.current_status = "down"
            if open_incident(db, monitor.id) is None:
                db.add(Incident(monitor_id=monitor.id, cause=check.error or "check failed"))
    db.add(check)
    db.commit()
    cache.invalidate(f"monitors:list:{monitor.owner_id}") # the only link back to the API
    cache.invalidate(f"monitors:checks:{monitor.id}:*")
    cache.invalidate("status:public:*")
```

backend/app/probe.py, lines 68-100 (condensed)

1.4 Low coupling — parts meet only at narrow, explicit seams

- **API and worker never call each other.** They share only the database schema and a naming convention for cache keys (last three lines above). Either one can be restarted, redeployed or scaled without the other noticing.
- **Frontend and backend share only a JSON contract.** Pydantic schemas (`MonitorOut`, `IncidentOut`) on one side are mirrored by TypeScript types in `api.ts` on the other. A type mismatch fails `next build` in CI, so contract drift cannot ship.
- **Cross-cutting concerns are injected, not constructed.** Routers receive a session and a verified user through `FastAPI Depends`. None of them parses a token or opens a connection, and ownership is enforced in one helper:

```
def _owned(db: Session, monitor_id: int, user: User) -> Monitor:
    monitor = db.get(Monitor, monitor_id)
    if monitor is None or monitor.owner_id != user.id:
        raise HTTPException(status_code=404, detail="No monitor with that id.") # 404, not 403: no existence
    return monitor

@router.get("")
def list_monitors(db: Session = Depends(get_db), user: User = Depends(current_user)): ...
```

backend/app/routers/monitors.py, lines 20-28

- **Configuration is one object.** `config.Settings` reads the environment once. No module calls `os.environ`, so the failure threshold, probe timeout and TTLs can all change without a code edit.

2 High-Level Architecture

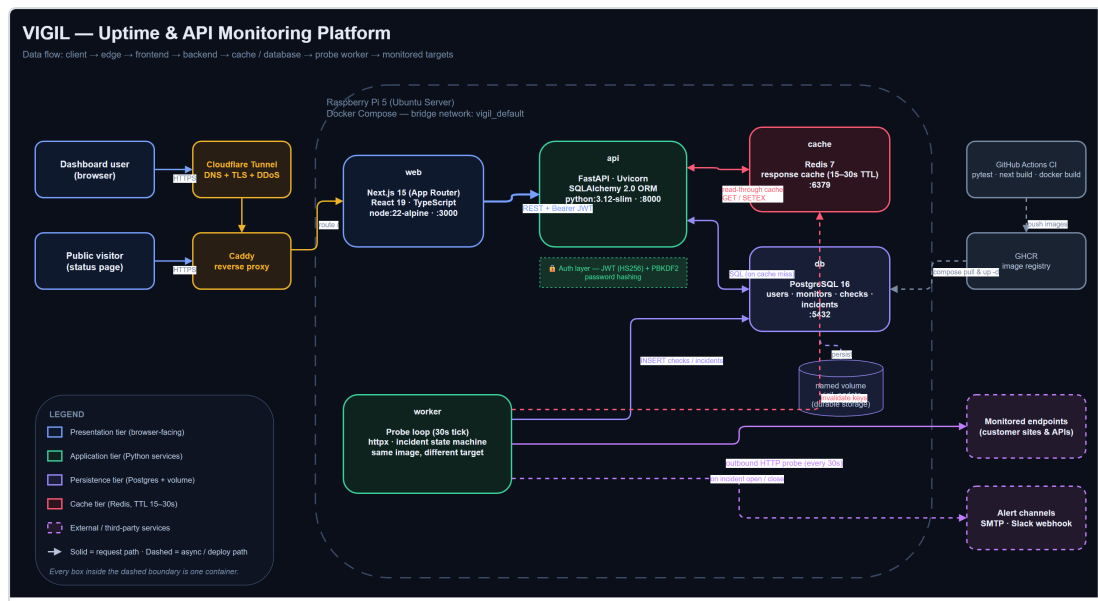


Figure 1 — System architecture: client → edge → frontend → API → cache / database, with the probe worker alongside. Editable source: docs/design/diagrams/01-architecture.drawio

2.1 Style chosen: a layered monolith, deployed as cooperating containers

Vigil is a **client-server system with a layered backend** (presentation → API → domain services → data access), packaged as five containers: web, api, worker, PostgreSQL and Redis. It is deliberately *not* microservices. The API is a single process with four routers over one database. The only thing split out is the worker, and that split has a concrete reason.

- **Why layered:** one developer, one semester. A strict downward dependency rule can be checked by reading the imports, and it keeps each change inside one layer.
- **Why a separate worker:** probing is slow, blocking I/O. A monitor that times out holds a thread for eight seconds. If that happened inside the API, the dashboard would slow down during an outage, exactly when people are reloading it.
- **Why not microservices:** they would add network hops, distributed failure modes and more deployment work, with nothing to show for it at this size. Because the module seams already exist, a service could be split out later along a line that is already there.
- **Why cache-aside with write-invalidation:** the public status page spikes exactly when a service is failing. With a 20-second TTL, a thousand visitors cost about three database queries a minute. The worker deletes the affected keys on every write, so data is never staler than one probe cycle.

The system diagram is unchanged since Review 1 because the implementation matched it. The **update for this review** is three new design views that go one level deeper: modules (Figure 2), data (Figure 3) and behaviour (Figure 4).

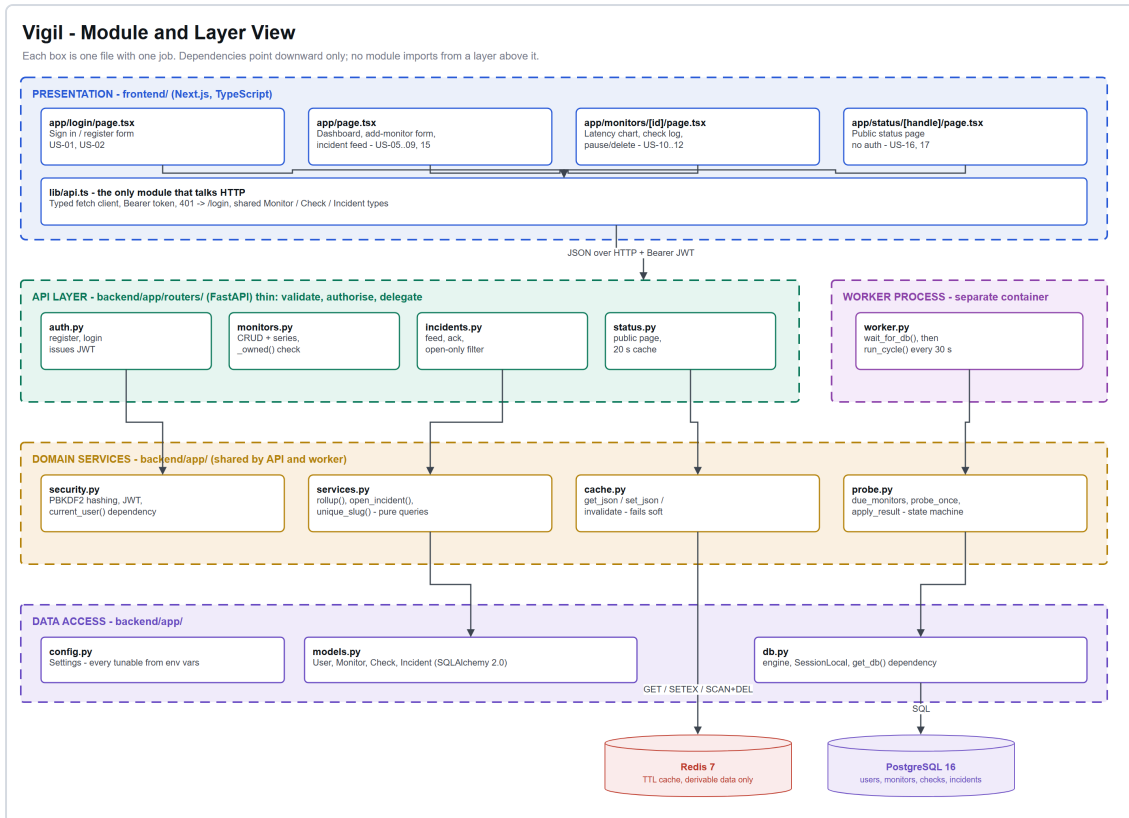


Figure 2 — Module and layer view. Each box is one file; arrows are imports and point downward only.

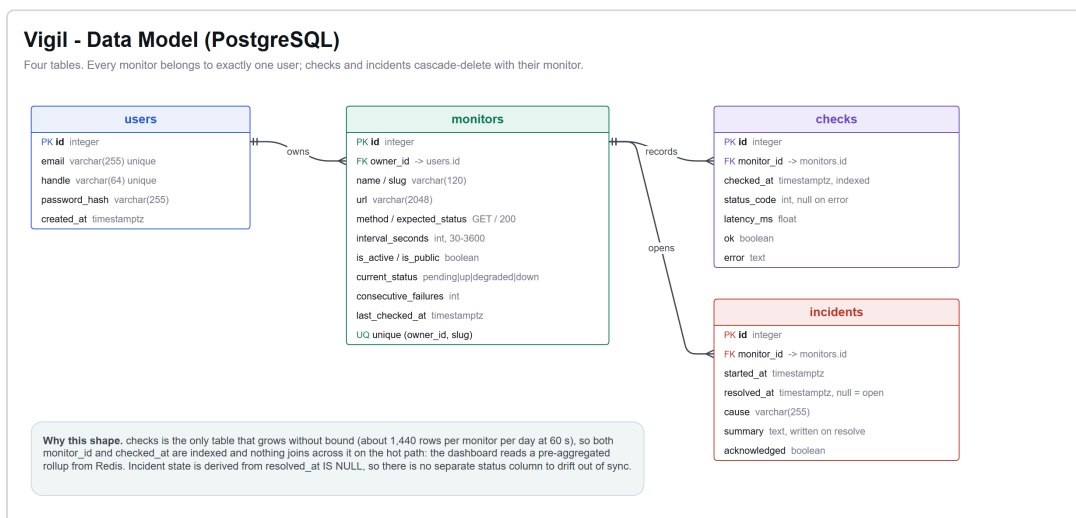


Figure 3 — Data model. Checks and incidents cascade-delete with their monitor; an incident is open exactly when `resolved_at` is null, so there is no second status column to fall out of sync.

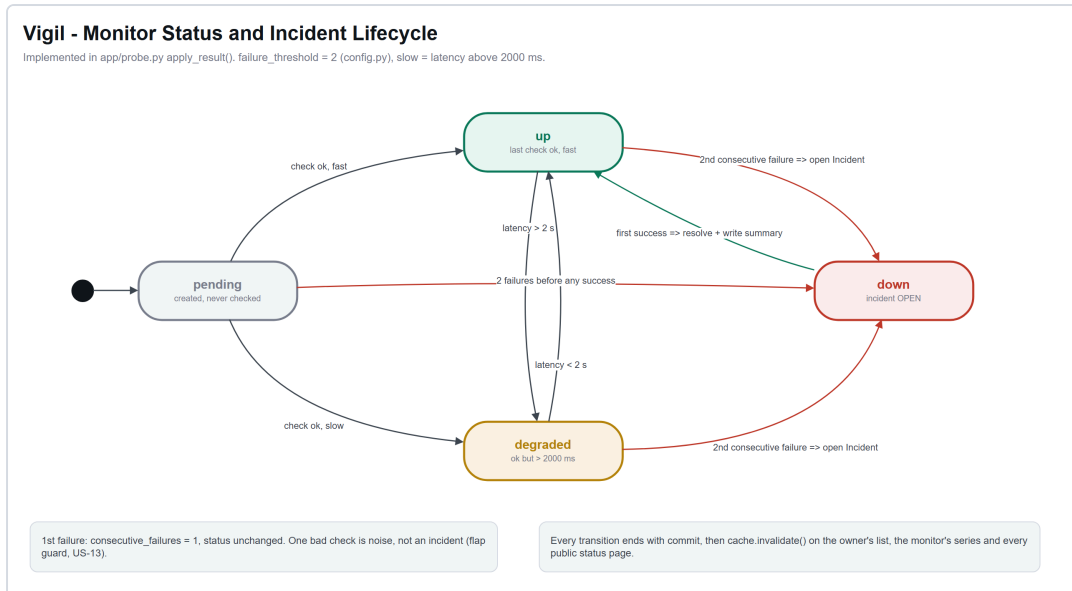
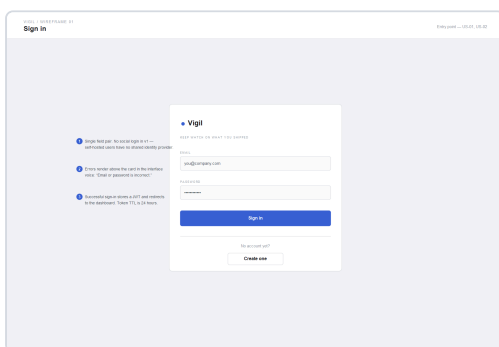


Figure 4 — Monitor state machine implemented by `apply_result()`. A single failure changes nothing (flap guard); the second opens an incident; the first success closes it.

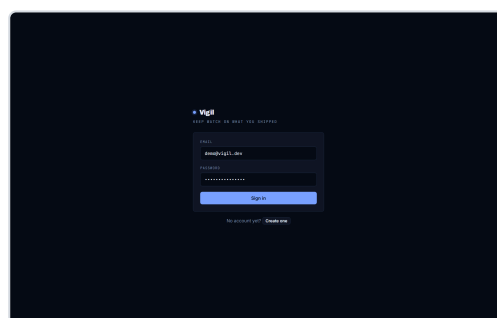
3 User Interface Design

Each of the six screens from Review 1 is shown as its wireframe (left) next to the implemented screen captured from the running application (right). The wireframes are SVG files in `docs/design/ui/wireframes/`. Dragging them into a Figma file opens them as editable vector frames.

WIREFRAME

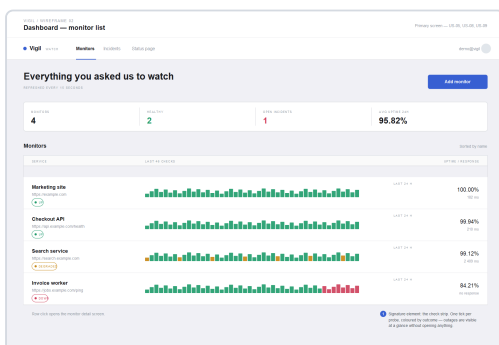


IMPLEMENTED

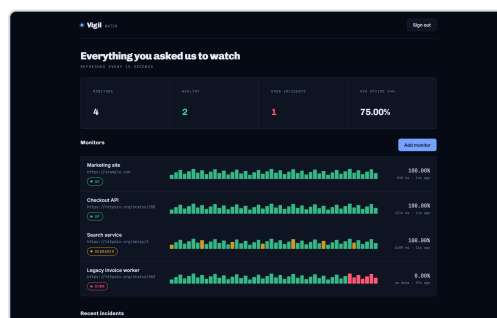


1 • Sign in (US-01, US-02). One card, two fields, one primary action; "Create one" switches the same form to registration instead of sending users to another page.

WIREFRAME

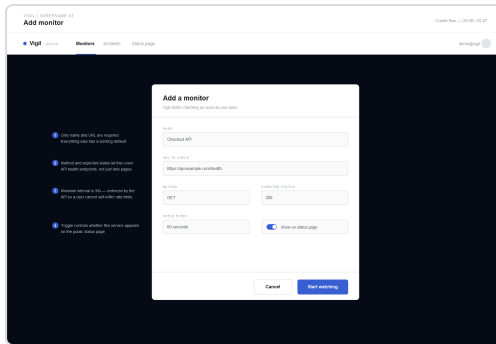


IMPLEMENTED

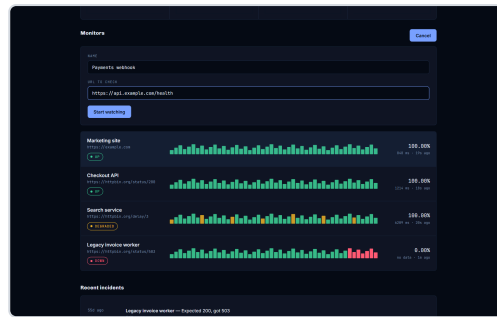


2 • Dashboard (US-05, US-08, US-09). Four summary tiles answer "is anything broken?" at a glance. Each row shows status, 24-hour uptime and latency, and the page refreshes itself every 15 seconds.

WIREFRAME

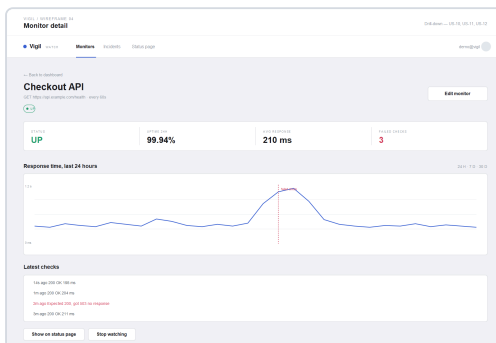


IMPLEMENTED

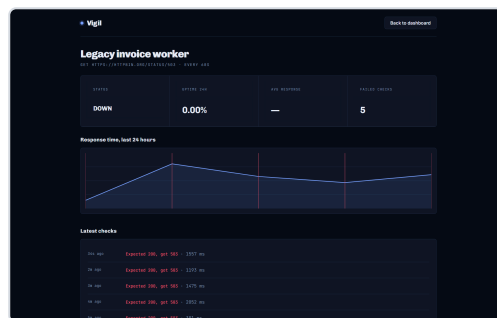


3 • Add monitor (US-06, US-07). Only a name and a URL are required. Method, expected status and interval have working defaults, and the server clamps the interval to 30–3600 s with a readable error.

WIREFRAME

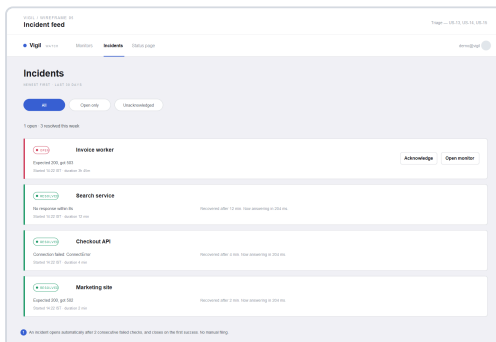


IMPLEMENTED

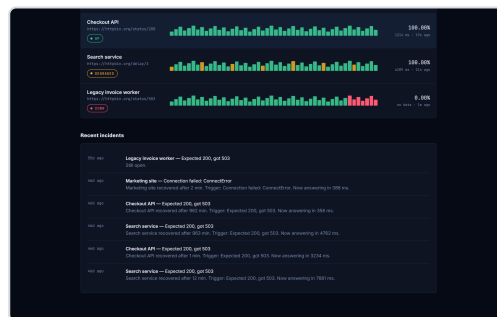


4 • Monitor detail (US-10, US-11, US-12). Failed checks are drawn as red vertical markers on the chart, so a gap is explained rather than just missing. The check log shows what the server actually returned.

WIREFRAME

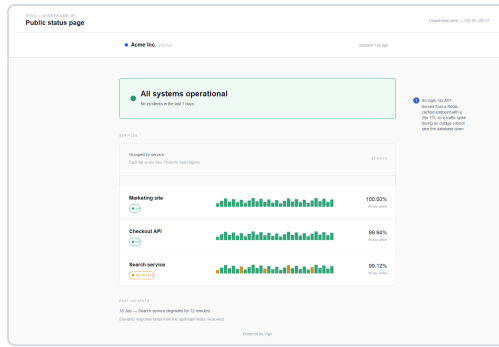


IMPLEMENTED

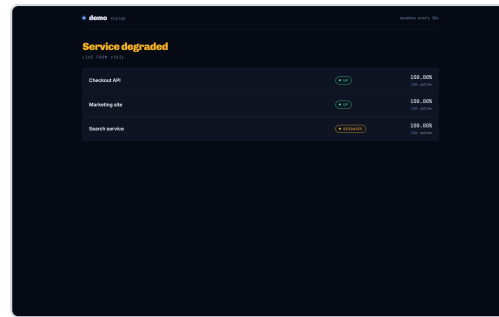


5 • Incident feed (US-13, US-14, US-15). *Changed from the wireframe:* the feed moved from its own page to the bottom of the dashboard, which saves a navigation step. Open incidents are pinned to the top, and each resolved one carries its plain-English summary.

WIREFRAME



IMPLEMENTED



6 • Public status page (US-16, US-17). No login. One banner derived from the worst service status, and only monitors the owner marked public. URLs and raw errors are never shown.

3.1 How the UI was made user-friendly

- **Status is never shown by colour alone.** Every status pill carries its word (UP / DEGRADED / DOWN) next to the coloured dot, so it still reads correctly for colour-blind users and in greyscale print.
- **Plain language instead of exception names.** Users see "Expected 200, got 503" and "No response within 8s", never `httpx.ConnectTimeout`. Resolved incidents read as a sentence a non-engineer can forward to customers (US-21).
- **Few decisions, good defaults.** Adding a monitor takes two fields. Everything else has a sensible value that can be changed later.
- **Empty and error states tell the user what to do next.** With no monitors, the dashboard explains how to add one. If the API is unreachable, an inline alert appears and polling continues rather than leaving a blank page.
- **One visual language on all six screens.** The same tokens (colour, type, spacing) come from `globals.css`. Monospace is used only for machine data (URLs, latencies, timestamps), so it is easy to tell what the user typed from what the system measured. Keyboard users get a visible focus ring.

4 Design Decisions & Why

1 • The probe worker runs in its own container.

Probing is slow, blocking I/O. Keeping it out of the API process means an outage cannot slow down the tool that reports it.

Trade-off: two processes to run. Accepted, because Compose starts both with one command and they share one image.

2 • Redis is a cache-aside layer with short TTLs, invalidated on write, and it fails soft.

The status page and dashboard are the read-heavy endpoints, and they spike during incidents. The TTL puts a ceiling on staleness, invalidation makes updates show up almost immediately, and swallowing Redis errors means the cache can never cause an outage of its own.

Trade-off: a `SCAN` on each write. Cheap at this key count; it would be revisited if the number of keys grew large.

3 • An incident opens after two consecutive failures, not one.

Single-check alerting fills the channel with false alarms until people mute it (the Arjun persona). The threshold is a setting in `config.py`, not a constant in the code.

Trade-off: detection takes up to one extra interval. This is exactly the "within twice the interval" success metric from Review 1.

4 • Ownership is enforced in one helper, and other users' data returns 404, not 403.

Account isolation (US-04) lives in `_owned()` and the `current_user` dependency rather than being repeated in every handler. A 404 does not reveal that the monitor exists.

Trade-off: none worth noting. It is also less code.

5 · One typed API client on the frontend, one settings object on the backend.

Every network call goes through `api<T>()` and every tunable goes through `Settings`, so changes to transport, auth or configuration each happen in one file.

Trade-off: TypeScript types copy the Pydantic schemas by hand. Generating them from the OpenAPI spec is a planned improvement.

5 Maintainability and Future Changes

A design shows its worth when a change arrives. Each planned change below lands in one place, and the right-hand column lists what stays untouched:

Future change	Where it lands	What does not change
Email and Slack alerts (US-19, US-20)	New <code>alerts.py</code> , called from the two transition points in <code>apply_result()</code>	Routers, frontend, schema
TCP, DNS or keyword checks	A <code>kind</code> column plus one probe strategy per kind, chosen inside <code>probe_once()</code>	The state machine, which only sees a <code>Check</code>
Check retention and downsampling	A periodic job in the worker loop	<code>rollup()</code> and every reader of it
Multi-region probing	Run <code>N</code> worker containers, each with a region label	API and frontend
Token in an <code>httpOnly</code> cookie	<code>lib/api.ts</code> and <code>security.py</code>	All four pages

5.1 A defect this review caught, and why the fix was one file

Capturing the screens for this document exposed a bug. The dashboard counted open incidents from the incident feed, but the feed is capped at the newest 100 rows. An incident that stayed open long enough fell off the list, and the "Open incidents" tile read **0** while a monitor was down. The API already had an `open_only` filter, so the fix touched only the page that consumed it:

```
// Open incidents are fetched on their own: the feed is capped at the
// newest 100, so an old incident that is still open can fall off it.
const [m, i, open] = await Promise.all([
  api<Monitor[]>("/api/monitors"),
  api<Incident[]>("/api/incidents"),
  api<Incident[]>("/api/incidents?open_only=true"),
]);
setIncidents([...open, ...i.filter((x) => x.resolved_at)]);
```

frontend/app/page.tsx, lines 48–56 (condensed)

5.2 Known design debt, stated plainly

- **The dashboard check strip is illustrative.** It is currently drawn from the monitor's status, not from real check history. The next step is to feed it the last 48 checks from the existing series endpoint.
- **No migration tool yet.** Tables are created with `create_all()`. Alembic has to be adopted before the first schema change after release.
- **Thin test suite.** Only hashing and slug generation are tested today. Because `probe_once()` and `apply_result()` were kept free of I/O coupling, the state machine in Figure 4 can be tested table-style without a network.

6 Repository Updates

```
docs/design/
├── README.md                    index of this folder
├── diagrams/
│   ├── 01-architecture.drawio + .png
│   ├── 02-module-layers.drawio + .png
│   ├── 03-data-model.drawio   + .png
│   └── 04-monitor-state-machine.drawio + .png
├── ui/
│   ├── wireframes/           6 x .svg (Figma-importable) + .png
│   └── screens/               6 x .png captured from the running app
└── Vigil_Software_Design_Document.pdf
```

- **Editable sources:** every diagram is a `.drawio` file that opens at app.diagrams.net.

The four new ones are generated by `scripts/make_design_diagrams.py`.

- **PNG exports:** `scripts/export_diagrams.py` renders each `.drawio` with the official `diagrams.net` viewer, so every PNG matches its source exactly.

- **README.md:** new section "*Software Design*" embeds the diagrams and summarises the main design choices.

- **Link:** github.com/vatsal-agra/vigil/tree/main/docs/design